



SQL Basics for Beginners

Brief Overview

This note covers **SQL** and was created from the [SQL Course for Beginners \[Full Course\]](#) YouTube video, 190 minutes long. It guides you through creating and managing databases with MySQL Workbench, learning how to **retrieve, insert, update, and delete** data, and exploring tables, relationships, joins, sub-queries, regex, and hands-on exercises.

Key Points

- 190-minute walkthrough of CRUD operations and database fundamentals.
 - Practical MySQL Workbench sessions with a sample database.
 - In-depth coverage of joins, regex, and advanced SQL features.
 - Interactive exercises and downloadable resources to reinforce learning.
-

Course Overview

- Learn to **retrieve, insert, update, and delete** data.
- Topics include tables, relationships, joins, sub-queries, regular expressions, and more.
- Hands-on exercises reinforce SQL syntax.
- Materials: cheat sheet, downloadable SQL files, and a table of contents for quick navigation.

What Is a Database?

A **database** is a collection of data stored in a format that can be easily accessed.

A **Database Management System (DBMS)** is software that manages databases, executes queries, and returns results.

Types of Database Management Systems

Relational (SQL)

In **relational databases**, data is stored in tables that are linked by relationships.

- Each table represents a specific entity (e.g., **customer, product, order**).
- SQL (Structured Query Language) is used to query and modify data.
- Popular RDBMS: **MySQL, SQL Server, Oracle** (all follow the standard SQL specification).

Non-Relational (NoSQL)

Non-relational databases do not use tables or relationships and have their own query languages.

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data model	Tables with rows & columns	Documents, key-value, graphs, etc.
Relationships	Explicit foreign keys	Usually none or implicit
Query language	SQL	Proprietary (e.g., MongoDB query)
Schema	Fixed (defined beforehand)	Flexible / schema-less
Typical use cases	Transactional systems, reporting	Big data, real-time analytics

SQL History & Pronunciation

- Developed at IBM in the 1970s as **SQUEL** (Structured English Query Language).
- Renamed to **SQL** because “SQUEL” was a trademark of an airplane company.
- Both pronunciations “**SQUE-el**” and “**SE-kwell**” are used; preference varies by region.

Installing MySQL (Community Edition)

macOS Installation

1. Download the **MySQL Community Server** DMG from *mysql.com* (Community Edition, free).
2. Open the DMG and run the installer package; follow the wizard (continue, accept license).
3. Set a **root** password when prompted.
4. Install **MySQL Workbench** (download DMG, drag to Applications).
5. Open Workbench, create a new connection:
 - Name: *local instance*
 - Host: 127.0.0.1
 - Port: 3306
 - Username: root
 - Store password in keychain and test the connection.

Windows Installation

1. Download the **MySQL Community Server** installer from *mysql.com*.
2. Run the installer; choose **Developer Default** setup.
3. Accept warnings (e.g., missing Python connector) and proceed.
4. Select **MySQL Server** and **MySQL Workbench** for installation; click **Execute**.
5. Set a **root** password when prompted.
6. After installation, open Workbench, add a new connection with default settings (host 127.0.0.1, port 3306, user root, stored password).

MySQL Workbench Setup

- Toolbar icons: create new SQL tab, open SQL file, create database, create tables.
- Navigator panel: **Administration** (server control) and **Schemas** (list of databases).
- Query editor: main area for writing SQL statements.
- Right-hand panels: **Contacts** and **Snippets** (optional).
- To hide panels, use the view toggle buttons.

Database Objects Overview

Tables store actual data in rows (records) and columns (fields).

Views are virtual tables built from queries, useful for reporting.

Stored procedures and **functions** are pre-written programs stored in the database for reusable logic.

Example Tables in the Sample Database

- customers – columns: customer_id, first_name, last_name, birth_date, phone, address.
- orders – columns: order_id, customer_id, order_date, status.
- products, shippers, etc.

Relationships Between Tables

- The customer_id column in **orders** references customer_id in **customers**.
- This foreign-key relationship avoids data duplication; changes to a customer's info are made in one place (the **customers** table).

Creating the Course Databases

1. Download the provided ZIP file (contains SQL scripts).
2. Open create_databases.sql in Workbench.
3. Execute the script with the **yellow thunder** icon (runs the whole file if nothing is selected).
4. Refresh the **Schemas** tab to see the newly created databases (prefixed with SQL_).

Basic SELECT Query Syntax

The **SELECT** statement retrieves data from a table.

```
SELECT column1, column2 FROM table_name;
```

- Use * to select **all columns**: SELECT * FROM customers;
- End each statement with a **semicolon** (;).
- SQL keywords are case-insensitive; convention is uppercase for keywords, lowercase for identifiers.

Clause Order (mandatory sequence)

1. **SELECT** – list of columns or *.
2. **FROM** – source table.
3. **WHERE** – optional filter condition.
4. **ORDER BY** – optional sorting.

Incorrect order results in a **syntax error**.

Optional Clauses

- **WHERE** filters rows:

```
SELECT * FROM customers WHERE customer_id = 1;
```

- **ORDER BY** sorts results:

```
SELECT * FROM customers ORDER BY first_name;
```

- **Comments**:

- Single-line comment begins with --.

```
-- This line is ignored by the SQL engine SELECT * FROM customers;
```

Executing Queries in Workbench

1. Choose the target database (e.g., USE SQL_store;).
2. Write the query in the editor; press **Shift + Command + Enter** (macOS) or the **Execute** button.
3. Results appear in the output pane; errors are highlighted with red underlines.

Exploring the Sample Database

- Double-click a schema to make it bold (active).
- Expand **Tables** to view column definitions.
- Right-click a table and select the **lightning bolt** icon to preview data.
- Observe how primary keys (e.g., customer_id) uniquely identify records and serve as foreign keys in related tables.

SELECT Clause – Expressions & Aliases

Expression – a combination of column names, constants, and arithmetic operators that MySQL evaluates for each row.

- Columns can be listed **explicitly** (e.g., first_name, last_name) instead of using *.
- Arithmetic operators available:
 - + addition - subtraction * multiplication / division % modulus (remainder)

Arithmetic Expressions

```
SELECT first_name,  
       last_name,  
       points,  
       points + 10 AS discount_factor  
FROM customers;
```

- The result shows the original points column and a new column where **10** is added to each point total.

Operator Precedence & Parentheses

In arithmetic, **multiplication** and **division** have higher precedence than **addition** and **subtraction**.

- Example without parentheses (multiplication first):

```
SELECT points * 10 + 100 AS calc_value  
FROM customers;
```

- Changing the order with parentheses:

```
SELECT (points + 10) * 100 AS calc_value  
FROM customers;
```

- Use parentheses to **override** default precedence and to improve readability.

Column Aliases (AS)

Alias – a temporary name given to a column or expression in the result set.

- Syntax: expression AS alias_name
- Alias can contain spaces if enclosed in single or double quotes:

```
SELECT points + 10 AS "discount factor"
FROM customers;
```

DISTINCT Keyword

DISTINCT eliminates duplicate rows from the result set.

```
SELECT DISTINCT state
FROM customers;
```

- Returns each state only once, even if multiple customers share the same state.

WHERE Clause – Filtering Data

WHERE restricts rows returned by a query based on a **condition** that evaluates to true.

Comparison Operators

Operator	Meaning
>	greater than
>=	greater than or equal
<	less than
<=	less than or equal
=	equal
!= or <>	not equal

- Example – customers with more than 3000 points:

```
SELECT *
FROM customers
WHERE points > 3000;
```

String & Date Literals

- Enclose **strings** and **dates** in single quotes ('...'):

```
WHERE state = 'Virginia'
```

```
WHERE birth_date = '1990-01-01'
```

Logical Operators (AND, OR, NOT)

- AND both conditions must be true.
- OR at least one condition must be true.
- NOT negates a condition.

Combining Conditions

```
SELECT *  
FROM customers  
WHERE birth_date > '1990-01-01' AND points > 1000;
```

```
SELECT *  
FROM customers  
WHERE state = 'Virginia' OR points > 1000;
```

Operator Precedence

AND is evaluated **before** OR unless parentheses dictate otherwise.

- Without parentheses:

```
WHERE birth_date > '1990-01-01' OR points > 1000 AND state = 'Virginia';
```

→ points > 1000 AND state = 'Virginia' is evaluated first.

- With parentheses for intended logic:

```
WHERE birth_date > '1990-01-01' OR (points > 1000 AND state = 'Virginia');
```

Using NOT and De Morgan's Laws

- Negating a compound condition can be simplified:

```
-- Original  
SELECT *  
FROM customers  
WHERE NOT (birth_date > '1990-01-01' OR points > 1000);
```

- Apply De Morgan:
 - NOT (A OR B) → A AND NOT B
 - NOT (A AND B) → A OR NOT B
- Simplified version:

```
SELECT *  
FROM customers  
WHERE birth_date <= '1990-01-01' AND points <= 1000;
```

IN Operator

IN tests whether a column's value matches **any** value in a list.

```
SELECT *  
FROM customers  
WHERE state IN ('Virginia', 'Florida', 'Georgia');
```

- Equivalent to multiple OR conditions but more concise.
- Excluding the list uses NOT IN:

```
WHERE state NOT IN ('Virginia', 'Florida', 'Georgia');
```

BETWEEN Operator

BETWEEN ... AND ... checks if a value lies within an **inclusive** range.

```
SELECT *  
FROM customers  
WHERE points BETWEEN 1000 AND 3000;
```

- Works with numbers, dates, and strings:

```
WHERE birth_date BETWEEN '1990-01-01' AND '2000-01-01';
```

LIKE Operator & Pattern Matching

LIKE matches a column against a **string pattern** using wildcards.

Wildcard	Meaning
%	any sequence of zero or more characters
_	exactly one character

Examples

- Names starting with **B**:

```
WHERE last_name LIKE 'B%';
```

- Names containing **brush**:

```
WHERE last_name LIKE 'brush%';
```

- Names that have an **e** anywhere:

```
WHERE last_name LIKE '%e%';
```

- Names ending with **y**:

```
WHERE last_name LIKE '%y';
```

- Exactly six characters, ending with **y**:

```
WHERE last_name LIKE '____y';
```

- Start with **b**, then exactly four characters, ending with **y**:

```
WHERE last_name LIKE 'b____y';
```

Combining with NOT

- Exclude a pattern:

```
WHERE phone NOT LIKE '%9';
```

Practical Exercises (Reference)

1. **Product pricing** – list name, unit_price, and a calculated new_price (10% increase):

```
SELECT name,  
       unit_price,  
       unit_price * 1.1 AS new_price  
FROM products;
```

2. **Orders placed this year** (assuming the current year is 2019):

```
SELECT *  
FROM orders  
WHERE order_date >= '2019-01-01';
```

3. **Order items with total cost > 30 for order 6:**

```
SELECT *  
FROM order_items  
WHERE order_id = 6  
      AND quantity * unit_price > 30;
```

4. **Customers living in specific states:**

```
SELECT *  
FROM customers  
WHERE state IN ('Virginia', 'Florida', 'Georgia');
```

5. **Products with stock quantity of 49, 38, or 72:**

```
SELECT *  
FROM products  
WHERE quantity_in_stock IN (49, 38, 72);
```

6. **Customers born between 1990-01-01 and 2000-01-01:**

```
SELECT *
FROM customers
WHERE birth_date BETWEEN '1990-01-01' AND '2000-01-01';
```

7. **Addresses containing “tray” or “avenue”:**

```
SELECT *
FROM customers
WHERE address LIKE '%tray%'
      OR address LIKE '%avenue%';
```

8. **Phone numbers ending with 9:**

```
SELECT *
FROM customers
WHERE phone LIKE '%9';
```

Regular Expressions (RegEx) in MySQL

RegEx – a pattern-matching syntax that lets you search for complex string structures beyond what LIKE can express.

- The operator is REGEXP (or its synonym RLIKE).
- Basic symbols:
 - ^ – start of a string
 - \$ – end of a string
 - | – logical OR between patterns
 - [...] – any single character inside the brackets
 - [...] with a hyphen (e.g., [a-h]) – a range of characters

Simple examples

```
-- Find last names that contain the word “field”
SELECT * FROM customers
WHERE last_name REGEXP 'field';
```

```
-- Last name must start with “field”
SELECT * FROM customers
WHERE last_name REGEXP '^field';
```

```
-- Last name must end with "field"
SELECT * FROM customers
WHERE last_name REGEXP 'field$';
```

```
-- Either "field" *or* "mac" appears in the last name
SELECT * FROM customers
WHERE last_name REGEXP 'field|mac';
```

```
-- "field", "mac", *or* "rows" appear in the last name
SELECT * FROM customers
WHERE last_name REGEXP 'field|mac|rows';
```

```
-- Must start with "field", *or* contain "mac", *or* contain "rows"
SELECT * FROM customers
WHERE last_name REGEXP '^field|mac|rows';
```

```
-- Must start with "field" **and** end with it
SELECT * FROM customers
WHERE last_name REGEXP '^field$';
```

```
-- An "e" preceded by **g**, **i**, **m**, or **a-h**
SELECT * FROM customers
WHERE last_name REGEXP '[gim]e';
```

```
-- An "e" followed by **f**, **m**, or **q**
SELECT * FROM customers
WHERE last_name REGEXP 'e[fmq]';
```

```
-- "e" preceded by any character from **a** to **h**
SELECT * FROM customers
WHERE last_name REGEXP '[a-h]e';
```

Quick recap of RegEx symbols

Symbol	Meaning
^	beginning of string
\$	end of string

[...]	any single character listed
[a-z]	any character in the range a-z

RegEx Practice Exercises

1. First names “Elka” or “Ambur”

```
SELECT * FROM customers
WHERE first_name REGEXP 'Elka|Ambur';
```

2. Last names ending with “ey” or “on”

```
SELECT * FROM customers
WHERE last_name REGEXP 'ey$|on$';
```

3. Last names that start with “my” or contain “se”

```
SELECT * FROM customers
WHERE last_name REGEXP '^my|se';
```

4. Last names containing “e” followed by “r” or “u”

```
SELECT * FROM customers
WHERE last_name REGEXP 'e[r|u]';
-- Equivalent using brackets:
-- WHERE last_name REGEXP 'e[ru]';
```

Handling Missing Values (NULL)

NULL – represents the absence of a value; it is **not** the same as zero or an empty string.

- Test for NULL with IS NULL or IS NOT NULL.
- Example: find customers without a phone number.

```
SELECT * FROM customers
WHERE phone IS NULL;
```

- Find customers **with** a phone number.

```
SELECT * FROM customers
WHERE phone IS NOT NULL;
```

- Orders that are not shipped (missing ship_date or shipper_id):

```
SELECT * FROM orders
WHERE ship_date IS NULL
      OR shipper_id IS NULL;
```

Sorting Results (ORDER BY)

- By default MySQL returns rows ordered by the **primary key** (e.g., customer_id).
- Syntax: ORDER BY column_name [ASC|DESC].

Single-column sorting

```
SELECT * FROM customers
ORDER BY first_name ASC;  -- alphabetical
```

Descending order

```
SELECT * FROM customers
ORDER BY first_name DESC;
```

Multiple columns

```
SELECT * FROM customers
ORDER BY state ASC, first_name ASC;
```

Sorting by a column **not** in the SELECT list (MySQL-specific)

```
SELECT first_name, last_name
FROM customers
ORDER BY birth_date DESC;
```

Sorting by an alias or expression

```
SELECT *, points + 10 AS bonus
FROM customers
ORDER BY bonus DESC, first_name ASC;
```

Positional ordering (avoid)

```
-- NOT recommended: ORDER BY 1,2
SELECT first_name, last_name
FROM customers
ORDER BY 1,2;
```

Using column names prevents breakage when the SELECT list changes.

Limiting Result Sets (LIMIT)

- LIMIT row_count returns at most row_count rows.
- LIMIT offset, row_count skips offset rows, then returns row_count rows (useful for pagination).

First 3 customers

```
SELECT * FROM customers
LIMIT 3;
```

Paginating: page 3 with 3 rows per page

```
SELECT * FROM customers
LIMIT 6, 3;  -- skip 6 rows, then fetch 3
```

Exercise – top 3 loyal customers (most points)

```
SELECT *
FROM customers
ORDER BY points DESC
LIMIT 3;
```

Clause order matters: SELECT ... FROM ... [WHERE ...] [ORDER BY ...] LIMIT Changing the order yields a syntax error.

Joining Tables (JOIN)

- Separate tables avoid data duplication; related rows are linked via **foreign keys**.
- INNER JOIN returns rows where the join condition is true.

Basic join syntax

```
SELECT *
FROM orders
JOIN customers
  ON orders.customer_id = customers.customer_id;
```

Selecting specific columns

```
SELECT orders.order_id,
       customers.first_name,
       customers.last_name
FROM orders
JOIN customers
  ON orders.customer_id = customers.customer_id;
```

Resolving ambiguous column names

- When two tables contain the same column (e.g., customer_id), prefix with the table name:

```
SELECT orders.order_id,
       orders.customer_id AS order_customer_id,
       customers.customer_id AS customer_customer_id,
       customers.first_name
FROM orders
JOIN customers
  ON orders.customer_id = customers.customer_id;
```

Table aliases for brevity


```
SELECT o.order_id,
       c.first_name,
       c.last_name
FROM orders AS o
JOIN customers AS c
  ON o.customer_id = c.customer_id;
```

Joining order_items with products

```
SELECT oi.order_id,
       oi.product_id,
       p.product_name,
       oi.quantity,
       oi.unit_price
FROM order_items AS oi
JOIN products AS p
  ON oi.product_id = p.product_id;
```

- Note: order_items.unit_price stores the price **at the time of the order**; products.unit_price reflects the **current** price.

Cross-Database Joins (brief)

- MySQL allows referencing tables from different databases by prefixing the table name with the database name (e.g., db2.products).
- The same join syntax applies; just qualify the table with its database name.

Cross-Database Joins

Cross-database join – a join that references tables residing in different databases by prefixing the table name with its database name (database_name.table_name).

- The **current database** (selected with USE database_name;) does not need a prefix.
- Only tables **outside** the current database require the database_name. prefix.

```
USE SQL_store;

SELECT *
FROM order_items AS oi
JOIN SQL_inventory.products AS p
  ON oi.product_id = p.product_id;
```

- Switching the current database changes which tables need a prefix.

```
USE SQL_inventory;

SELECT *
FROM SQL_store.order_items AS oi
JOIN products AS p
  ON oi.product_id = p.product_id;
```

Key point: Prefix only the tables that are not part of the active database.

Self Join – Joining a Table with Itself 🤝

Self join – a join where a table is linked to another instance of the same table, typically using different aliases.

- Common for hierarchical data (e.g., employees → managers).
- The column that references the same table is often called a **foreign-key self-reference** (e.g., reports_to).

```
USE SQL_hr;

SELECT e.employee_id,
       e.first_name AS employee_name,
       m.first_name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m
  ON e.reports_to = m.employee_id;
```

- Use **different aliases** (e for employee, m for manager) and qualify each column with its alias.
 - Aliasing result columns (e.g., AS manager_name) avoids duplicate column names.
-

Joining More Than Two Tables +

Multi-table join – chaining several JOIN clauses to combine three or more tables in a single query.

- Example: retrieve order information together with customer data and order status.

```
USE SQL_store;

SELECT o.order_id,
       o.order_date,
```

```

        c.first_name,
        c.last_name,
        os.name AS status
FROM orders AS o
JOIN customers AS c
    ON o.customer_id = c.customer_id
JOIN order_statuses AS os
    ON o.status = os.order_status_id;

```

- Order of JOINS does not affect the result set, but a clear aliasing scheme improves readability.
- When many tables are involved, **LEFT JOIN** may be needed for optional relationships (e.g., shipper information).

Composite Primary Keys

Composite primary key – a primary key that consists of two or more columns together, ensuring each row is uniquely identified by the combination.

- Typical in junction tables such as `order_items` where neither `order_id` nor `product_id` alone is unique.
- Design view shows a yellow key icon on each column that participates in the composite key.

```
-- order_items has a composite PK (order_id, product_id)
```

Joining tables that use composite keys

```

USE SQL_store;

SELECT oi.*,
       oin.note
FROM order_items AS oi
JOIN order_item_notes AS oin
    ON oi.order_id = oin.order_id
   AND oi.product_id = oin.product_id;

```

- Both columns must be included in the ON clause (a **compound join condition**).

Implicit Join Syntax (Deprecated)

Implicit join – specifying multiple tables in the FROM clause and placing the join condition in the WHERE clause.

```
SELECT *
FROM orders o, customers c
WHERE o.customer_id = c.customer_id;
```

- Functionally equivalent to an explicit INNER JOIN.
- **Risk:** forgetting the WHERE clause produces a **cross join** (Cartesian product), often returning far more rows than intended.
- Recommended practice: **always use explicit join syntax** (JOIN ... ON ...) to make the join condition mandatory.

Outer Joins (Left / Right)

Outer join – a join that returns all rows from one (or both) tables, inserting NULL for missing matches.

Left Join

- Returns **all rows from the left table** plus matching rows from the right table.
- Useful when you need to keep rows that may lack related records (e.g., customers without orders).

```
SELECT c.customer_id,
       c.first_name,
       o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```

- Rows with no matching order show NULL in order_id.

Right Join

- Returns **all rows from the right table**.
- Equivalent results can be achieved by swapping table positions and using a left join.

```
-- Same outcome as left join above, but with tables swapped
SELECT o.order_id,
       c.customer_id,
       c.first_name
FROM orders AS o
RIGHT JOIN customers AS c
  ON o.customer_id = c.customer_id;
```

- The OUTER keyword (LEFT OUTER JOIN, RIGHT OUTER JOIN) is optional.

Outer Join with Multiple Tables

- Combine left and inner joins to preserve rows from a primary table while filtering related tables.

```
SELECT o.order_id,  
       o.order_date,  
       c.first_name,  
       sh.name AS shipper,  
       os.name AS status  
FROM orders AS o  
LEFT JOIN customers AS c  
  ON o.customer_id = c.customer_id  
LEFT JOIN shippers AS sh  
  ON o.shipper_id = sh.shipper_id      -- keep orders even if shipper is missing  
JOIN order_statuses AS os  
  ON o.status = os.order_status_id;
```

- The left joins ensure all orders appear, regardless of whether a shipper exists; the inner join to order_statuses is safe because every order has a status.

Best Practices for Writing Joins

Practice	Reason
Prefer explicit JOIN ... ON ... syntax	Guarantees a join condition; avoids accidental cross joins.
Use left joins instead of right joins	Keeps the query flow left-to-right, easier to read when chaining multiple joins.
Alias tables consistently (e.g., c for customers, o for orders)	Reduces repetition and clarifies column origins.
Prefix every column with its alias when names collide	Prevents ambiguity and runtime errors.
Give meaningful aliases to calculated columns (e.g., AS manager_name)	Improves result-set readability.
When joining across databases, prefix only the external tables	Keeps queries concise and clear.
For composite keys, include all key columns in the ON clause	Ensures accurate matching of rows.

Exercise Summaries

1. **Cross-Database Join** – Join order_items (SQL_store) with products (SQL_inventory) using database prefixes.
2. **Self Join** – List each employee with their manager's name from the employees table.
3. **Three-Table Join** – Combine orders, customers, and order_statuses to show order ID, date, customer name, and status.
4. **Composite Key Join** – Join order_items with order_item_notes on both order_id and product_id.
5. **Implicit vs. Explicit Join** – Rewrite an implicit join as an explicit INNER JOIN; observe the difference when the WHERE clause is omitted.
6. **Outer Join Exercise** – Produce a list of all products with the total quantity ordered, showing NULL for products never ordered (use a left outer join).
7. **Mixed Outer/Inner Join** – Retrieve all orders with customer name, shipper name (if any), and order status, using left joins for optional relationships and an inner join for mandatory status.

Self Join 🤝

Self join – a join where a table is linked to another instance of the same table, usually using different aliases.

- Typical use case: hierarchical data such as employees ↔ managers.
- Alias e for **employees**, alias m for **managers**.

```
SELECT e.employee_id,  
       e.first_name,  
       m.first_name AS manager  
FROM employees AS e  
LEFT JOIN employees AS m  
  ON e.reports_to = m.employee_id;
```

- Using an **INNER JOIN** returns only employees that have a manager.
- Switching to a **LEFT JOIN** adds the top-level employee (e.g., the CEO) with NULL in the manager column.

USING Clause ✅

USING – a shortcut for ON when the join columns have **exactly the same name** in both tables.

```
SELECT o.order_id,  
       c.first_name,  
       c.last_name,  
       sh.name AS shipper_name  
FROM orders AS o  
JOIN customers AS c  
  USING (customer_id)
```

```
LEFT JOIN shippers AS sh
  USING (shipper_id);
```

- The USING (column_name) list can contain **one or more** columns.
- It works for both **INNER** and **LEFT** joins.
- **Cannot** be used when column names differ (e.g., orders.status vs. order_statuses.order_status_id). In that case the traditional ON clause is required.

Composite Key Join with USING

Composite primary key – a primary key composed of two or more columns; the combination uniquely identifies each row.

```
SELECT *
FROM order_items AS oi
JOIN order_item_notes AS oin
  USING (order_id, product_id);
```

- Both order_id **and** product_id must be listed in the USING clause to match the composite key.

Natural Join

Natural join – joins two tables by **all columns that share the same name** automatically.

```
SELECT o.order_id,
       c.first_name
FROM orders AS o
NATURAL JOIN customers AS c;
```

- The engine infers the join condition; **no column list is written**.
- **Caution:** if unintended columns share a name, the join may produce unexpected results. Prefer explicit joins for clarity.

Cross Join

Cross join – produces the Cartesian product of two tables (every row of the first combined with every row of the second).

Explicit syntax

```
SELECT c.first_name AS customer,
       p.name       AS product
FROM customers AS c
CROSS JOIN products AS p
ORDER BY c.first_name;
```

Implicit syntax

```
SELECT c.first_name AS customer,
       p.name       AS product
FROM customers AS c, products AS p
ORDER BY c.first_name;
```

- **Use case:** pairing a table of **sizes** (small, medium, large) with a table of **colors** (red, blue, green) to generate all size-color combinations.

UNION Operator

UNION – combines the result sets of two (or more) SELECT statements into a single result set.

Example: active vs. archived orders

```
SELECT order_id,
       order_date,
       'active' AS status
FROM orders
WHERE order_date >= '2019-01-01'

UNION

SELECT order_id,
       order_date,
       'archive' AS status
FROM orders
WHERE order_date < '2019-01-01'
ORDER BY first_name;
```

- Both queries must return the **same number of columns** and compatible data types.
- Column names are taken from the **first** SELECT statement.

Example: customers and shippers


```
SELECT first_name
FROM customers
```

UNION

```
SELECT name
FROM shippers;
```

- Returns a single column containing all distinct names from both tables.

Table Column Attributes

Symbol	Meaning	Example (customers)
PK	Primary key (unique row ID)	customer_id
NN	NOT NULL (value required)	first_name
AI	AUTO-INCREMENT (auto numbers)	customer_id
DEFAULT	Default value if none supplied	points default 0

- **Data types**
 - INT – whole numbers, no decimals.
 - VARCHAR(50) – variable-length string up to 50 characters; stores only the actual length.
 - CHAR(50) – fixed-length string; pads with spaces to 50 characters (wastes space).
- Columns marked **NN** must receive a value on insert; columns without NN (e.g., birth_date, phone) may receive NULL.

Inserting a Single Row

Full-column insert (using DEFAULT and NULL)

```
INSERT INTO customers
VALUES (DEFAULT,      -- customer_id (auto-increment)
       'Jon',         -- first_name
       'Smith',       -- last_name
       '1990-01-01',  -- birth_date
       NULL,          -- phone (optional)
       '123 Main St', -- address
       'Los Angeles', -- city
       'CA',          -- state
       DEFAULT);      -- points (default 0)
```

Column-list insert (no need for DEFAULT/NULL keywords)

```
INSERT INTO customers (first_name, last_name, birth_date,  
                        address, city, state)  
VALUES ('Jon', 'Smith', '1990-01-01',  
        '123 Main St', 'Los Angeles', 'CA');
```

- Column order can be rearranged when the column list is provided.
- After execution, the customer_id is auto-generated (e.g., 11) and points defaults to 0.

Inserting Multiple Rows

Shippers table (single column)

```
INSERT INTO shippers (name)  
VALUES ('Shipper One'),  
        ('Shipper Two'),  
        ('Shipper Three');
```

Products table (three columns, auto-increment ID omitted)

```
INSERT INTO products (name, quantity_in_stock, unit_price)  
VALUES ('Product 1', 10, 1.95),  
        ('Product 2', 20, 2.50),  
        ('Product 3', 15, 3.75);
```

- Each set of parentheses represents one row.
- The product_id column (auto-increment) is filled automatically.

Inserting Hierarchical (Parent-Child) Data

Parent-child relationship – a one-to-many link where a row in the parent table (e.g., orders) can have multiple related rows in the child table (e.g., order_items).

Steps

1. **Insert the parent row** (only non-auto columns).

```
INSERT INTO orders (customer_id, order_date, status)
```

```
VALUES (1, '2019-01-02', 1);
```

2. Retrieve the auto-generated order_id with `LAST_INSERT_ID()`.

```
SELECT LAST_INSERT_ID();
```

3. **Insert child rows** using the retrieved id.

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (LAST_INSERT_ID(), 1, 1, 2.95),
      (LAST_INSERT_ID(), 2, 1, 5.50);
```

- The same `LAST_INSERT_ID()` can be called repeatedly within the same session to reference the newly created order.
- All four columns in `order_items` are required, so the column list can be omitted.

Copying Tables with `CREATE TABLE ... AS SELECT`

Subquery – a `SELECT` statement embedded inside another SQL statement.

Creating a Full Copy

```
CREATE TABLE orders_archive AS
SELECT * FROM orders;
```

- The new table receives **all column data** but **does not copy** primary-key, `AUTO_INCREMENT`, or other column attributes.
- After creation, `order_id` must be supplied explicitly when inserting new rows because it is no longer auto-incrementing.

Copying a Subset (Using a Subquery)

1. **Truncate the archive** (remove all rows).

```
TRUNCATE TABLE orders_archive;
```

2. **Insert only orders before 2019-01-01.**

```
INSERT INTO orders_archive
SELECT * FROM orders
WHERE order_date < '2019-01-01';
```

Exercise Example (Invoices Archive)

- Goal: create `invoices_archive` that contains only paid invoices and replaces `client_id` with the client's name.

```
CREATE TABLE invoices_archive AS
SELECT i.invoice_id,
       i.invoice_number,
       c.name      AS client_name,
       i.invoice_total,
       i.payment_total,
       i.invoice_date,
       i.due_date,
       i.payment_date
FROM invoices AS i
JOIN clients  AS c USING (client_id)
WHERE i.payment_date IS NOT NULL;
```

- The `USING (client_id)` clause automatically joins on the column that shares the same name in both tables.

Updating Data

Basic UPDATE Syntax

```
UPDATE invoices
SET payment_total = 10,
    payment_date  = '2019-03-01'
WHERE invoice_id = 1;
```

- **SET** assigns new values to one or more columns.
- **WHERE** limits the rows that are affected.

Using Expressions, DEFAULT, and NULL

```
UPDATE invoices
SET payment_total = DEFAULT,      -- uses the column's default value (0)
    payment_date  = NULL         -- explicit NULL assignment
WHERE invoice_id = 1;
```

```
UPDATE invoices
SET payment_total = invoice_total * 0.5,
```

```
    payment_date = due_date
WHERE invoice_id = 3;
```

- Arithmetic expressions can be placed directly in the SET clause.
- DEFAULT and NULL are keywords, not strings.

Updating Multiple Rows

```
UPDATE invoices
SET payment_total = 0,
    payment_date = NULL
WHERE client_id = 3;
```

- **Safe-update mode** (enabled by default in MySQL Workbench) blocks updates that affect multiple rows without an explicit key condition.
- To disable it: *Edit → Preferences → SQL Editor → Uncheck “Safe Updates”* and reconnect.

Using IN with Updates

```
UPDATE invoices
SET payment_total = payment_total + 5
WHERE client_id IN (3, 4);
```

- Omitting the WHERE clause updates **all rows** in the table (use with caution).

Exercise: Add Points to Older Customers

```
USE SQL_store;

UPDATE customers
SET points = points + 15
WHERE birth_date < '1990-01-01';
```

Updating with Subqueries

Updating via a Lookup Subquery

```
UPDATE invoices
SET payment_total = 10,
```

```
payment_date = '2019-03-01'
WHERE client_id = (SELECT client_id FROM clients WHERE name = 'my works');
```

- The subquery returns a single scalar value (the matching client_id).
- If the subquery could return **multiple rows**, replace = with IN.

```
UPDATE invoices
SET payment_total = payment_total + 5
WHERE client_id IN (
    SELECT client_id FROM clients WHERE state IN ('California', 'New York')
);
```

Best Practice: Preview Before Updating

```
SELECT * FROM invoices WHERE payment_date IS NULL;
```

- Run the SELECT first to verify the target rows, then execute the UPDATE.

Exercise: Flag Gold-Customer Orders

```
UPDATE orders
SET comments = 'gold customer'
WHERE customer_id IN (
    SELECT customer_id FROM customers WHERE points > 3000
);
```

Deleting Data

Basic DELETE Syntax

```
DELETE FROM invoices
WHERE invoice_id = 1;
```

- Without a WHERE clause, **all rows** are removed – extremely dangerous.

Deleting with a Subquery

```
DELETE FROM invoices
WHERE client_id = (
```

```
SELECT client_id FROM clients WHERE name = 'my works'
);
```

- The subquery identifies the client_id to match.

Restoring Databases to Their Original State

1. Open **File → Open SQL Script** in MySQL Workbench.
2. Load create_databases.sql (the script that creates all course databases).
3. Execute the script to rebuild the original schema and data.
4. Refresh the **Schemas** pane to see the restored databases.

Reference Tables

Concept	Example	Key Points
Parent-child insert	Insert order → LAST_INSERT_ID() → insert items	Retrieve generated PK, reuse in child inserts
Create-as-select	CREATE TABLE orders_archive AS SELECT * FROM orders;	Copies data only , not constraints
Subquery in INSERT	INSERT INTO orders_archive SELECT * FROM orders WHERE ...;	Allows conditional copies
Update with expression	SET payment_total = invoice_total * 0.5	Calculations can be done inline
Update with DEFAULT	SET payment_total = DEFAULT	Resets column to its defined default
Safe-update mode	Workbench blocks multi-row updates without key	Disable via preferences if needed
Delete with subquery	DELETE FROM invoices WHERE client_id = (SELECT ...);	Use lookup to target rows